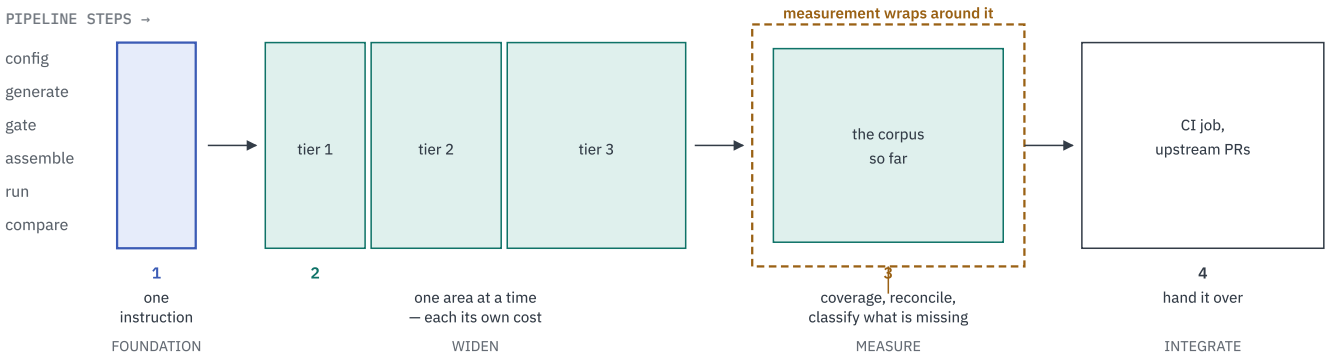
RISC-V SAIL TEST GENERATION · BUILD ORDER

What Gets Built, In What Order

Not by component, but by what you actually do: get one test working end to end, then widen it area by area across the model, then measure what it reached.

The shape of the work



The foundation is a vertical slice, not a horizontal layer.

We do not build all of generation, then all of running. We take one instruction through every step until it passes on two simulators. Everything after that is widening a pipe that already works — which is why the risk is front-loaded.

Measurement comes third because it has nothing to measure until there is a corpus. Building it earlier would mean a coverage report of almost nothing, which tells you neither that it works nor that the tests do.

Four stages. Only the first is all-or-nothing; the second is a repeated loop, and the third can start as soon as the second has produced anything worth measuring.

Stage 1 — Foundation

Goal: one instruction, generated and passing on two simulators. Until this works, nothing else can be tested, so it is done first and done narrow.

1. **Build the model twice.** The ordinary simulator, and a second build with coverage instrumentation. Kept separate so a coverage run can never silently use an uninstrumented emulator.
2. **Add the two entry points to the model.** `isla_testgen_init` and `isla_testgen_step`, preserved against dead-code elimination, with a post-build check that they survived. Proposed upstream from here.
3. **Compile the model to IR.** One `isla-sail` run per configuration. XLEN is a runtime selector; VLEN is baked in here.
4. **Fix what stops *any* RISC-V test.** The missing primitives the model calls, and the hardcoded 64-bit address width that mis-sizes RV32 reads. Not extension-specific — nothing runs without them.
5. **Implement the target and the ELF emitter.** The generator's target interface, plus the three portability assumptions that are true only of the architectures it already supports.
6. **Get one `addi` through, symbolically.** Generate, assemble with the real toolchain, run on Sail, run on Spike.
7. **In parallel, the oracle path.** Parse one Sail file, build a probe, run it, bake the values in, assert them. This depends on none of steps 2–5.

EXIT CRITERION

`addi x1, x0, 1` generates from both backends, and passes on Sail *and* Spike. Not “generation completes” — passes.

This stage carries almost all the technical risk. Every unknown — does the engine handle this model, do the entry points survive, does the emitter produce a loadable ELF — is resolved here, on one instruction, before any breadth is attempted. If something is going to be impossible, it becomes visible in stage 1 rather than in week ten.

Stage 2 — Widen, area by area

The machinery now works. Each area of the model is an increment, and each follows the same recipe:

1. Point the generator at that area — a set of Sail files for an extension, or a piece of state the preamble has to construct for the core.
2. Teach the renderer any operand kind it has not seen (a vector mask, a rounding mode, a CSR name).
3. Generate the sweep and run it.
4. Triage every failure into **model fault** or **our fault**. This is the step that takes the time.
5. Fix ours; write up theirs with a reproducer.
6. Record the area as done, with its pass count, its coverage and its exclusions.

“Extensions” is only two thirds of the model. Of the 14,975 instrumented spans, 10,437 are in `extensions/` — the rest is the core machinery every instruction runs through. Both have to be reached, and they are reached differently: an extension is widened by pointing the parser at more Sail files, the core by constructing the state that makes a branch in it taken.

What the model actually contains

AREA	SPANS	FILES	WHAT LIVES THERE	REACHED BY
extensions/	10,437	33	Every ISA extension, from I to vector crypto	Parsing its Sail files and generating instructions
core/	1,458	32	Register file, CSR dispatch, address checks, arithmetic helpers, `misa`, XLEN/FLEN/VLEN types, the SoftFloat interface	Executing instructions that call into it — mostly indirect
sys/	1,368	16	Virtual memory (PTE, page-table walk, TLB), PMA, the platform devices, reservations	Constructing page tables and memory attributes in the preamble
postlude/	1,356	11	Fetch, the step loop, CSR-clause merging, config validation, device tree	Running anything at all — plus config variation for the rest
pmp/	226	2	PMP entry matching and permission checks	Programming PMP entries, then accessing across their boundaries
prelude/, exceptions/, mops/, unit_tests/	129	7	Shared helpers, the exception taxonomy, memory operations	Faulting deliberately, and by ordinary execution

Tier 1 — the recipe is enough

TARGET	WHAT IT NEEDS	EACH
I, M, A, C, Zicsr, Zifencei, Zicond, Zawrs, Zihint*	Register and immediate operands only. Nothing new	hours
`core/` arithmetic and register helpers	Nothing directly — these are covered as a side effect of tier 1, because every instruction calls them	free

Tier 2 — a new operand kind or a new probe

TARGET	WHAT IT NEEDS BEYOND THE RECIPE	EACH
B, K (Zb*, Zk*)	Bit-manipulation operand forms; a separate backend, since the symbolic path cannot reach them	days
F, D (Zf*, Zd*, Zh*)	An FP probe. More results than callee-saved registers, so values go to a memory buffer before printing. Rounding modes become an operand kind. Also the only way to reach `core/softfloat_interface.sail` and `float_classify.sail`	days
Zicbom, Zicbop, Zicboz, Svinval, Zihpm, Zicntr	Cache-block, fence and counter operands. Small, but each its own shape	days
`postlude/` config paths	Nothing new to generate — but reaching `validate_config.sail` and the step loop's alternatives means *running the suite under more than one configuration*	days

Tier 3 — needs machinery that does not exist yet

TARGET	WHAT IT NEEDS FIRST	COST
V, vector_crypto, Zvabd plus core/vlen.sail	A vector probe, VLEN pinned to what the IR was built for, and legality constraints on every generated instruction — SEW, LMUL, register-group alignment, mask-register overlap	weeks
`pmp/` — 226 spans	A preamble that programs entries in the right order with lock bits set, then accesses across each boundary. Needs the engine's struct-typed vector fix before it can be done symbolically at all	weeks
`sys/` virtual memory vmem_ptw, vmem_pte, vmem_tlb	A preamble that builds a real page table and points `satp` at it. Every PTE feature and every fault class is a separate constructed state — this is the deepest single area in the model	weeks
`sys/pma.sail`	Region attributes come from the configuration JSON, so a PMA test is an ordinary access run under a configuration whose region declares it illegal — a config-generation step, not an instruction	days
Traps, interrupts, delegation exceptions/, core/interrupt_regs	The trap handler and the preamble that makes an interrupt deterministic rather than awaited	weeks
The 29 privileged targets	**The preamble builder**, which the four rows above are all instances of. Almost none map to an instruction	weeks

> **The privileged tier is the point of the engagement, and it is last in this stage for a reason.** A privileged scenario is an instruction executed in a constructed state. The instruction half has to work before the state half is worth building — otherwise a failure could be either, and triage becomes guesswork.

> **Two areas are never reached by generating instructions.** `postlude/device_tree.sail` and the RVFI files in `core/` exist for tracing and platform description, not for execution. They are excluded with that reason rather than counted as a gap — and the exclusion is derived from the model, not hand-listed.

Stage 3 — Measure what it reached

Only meaningful once there is a corpus. Built in this order because each step needs the one before it:

1. **Take the span manifest** from the instrumented build — every branch the model declares. This is the denominator, and it comes from the model rather than from us.
2. **Replay the corpus** under instrumentation, one test at a time, so a hit is attributable to the test that caused it.
3. **Reconcile.** Every span lands in exactly one of hit, reachable-but-missed, or excluded. A span in none of them is an accounting bug, not a result.
4. **Derive the exclusions** rather than listing them. Line numbers move; a checked-in list of line numbers silently rots into excluding the wrong code.
5. **Classify what is missing** by what it would take: a privilege level, a CSR field, a different configuration, an operand value.
6. **Publish that list** as the feedback interface, and re-enter generation.

> **One property to know before reading any coverage number.** A failing test writes no coverage data at all. Every negative control is therefore invisible to this stage — which is correct, but means a coverage gap and a failing test look nothing alike and must be read together.

Stage 4 — Hand it over

1. **Split the CI into two jobs.** A fast per-PR replay of the checked-in suite, and a slower scheduled job that regenerates and re-measures. Generation is solver-bound and must never gate a pull request.
2. **Submit the model changes upstream.** Complete only when merged, on a cadence this project does not set.
3. **Write the developer documentation** for what a maintainer cannot infer: how an extension is classified into the routing table, how a backend is replaced, why each exclusion exists.

Why this order

QUESTION	ANSWER
Why one instruction first?	Every technical unknown resolves on the narrowest possible case. Breadth added before the pipe works produces failures nobody can attribute
Why not measure earlier?	Coverage of almost nothing tells you neither that the measurement works nor that the tests do
Why privileged last within stage 2?	A privileged test is an instruction in a constructed state. If the instruction half is unproven, a failure could be either half
Why is the oracle path parallel?	It needs no solver, no IR and no upstream change. It is the schedule's safety margin: if the symbolic path slips, FP and vector still land
Why CI last?	There is nothing to replay until a suite exists, and its acceptance depends on maintainer review rather than on our effort

Extension names taken from the model's own `extensions/` directory; tiers reflect what each demands of the generator beyond the common recipe.